

Experiment 8

1) Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;
    // 1. malloc()
    int *a = (int *)malloc(3 * sizeof(int));
    if (a == NULL) {
        printf("malloc failed\n");
        return 1;
    }
    printf("Memory allocated using malloc()\n");
    for (i = 0; i < 3; i++) {
        a[i] = (i + 1) * 10;
        printf("a[%d] = %d\n", i, a[i]);
    }

    // 2. calloc()
    int *b = (int *)calloc(3, sizeof(int));
    if (b == NULL) {
        printf("calloc failed\n");
        free(a);
        return 1;
    }
    printf("\nMemory allocated using calloc()\n");
    for (i = 0; i < 3; i++) {
        printf("b[%d] = %d\n", i, b[i]);
    }

    // 3. realloc()
    a = (int *)realloc(a, 5 * sizeof(int));
    if (a == NULL) {
        printf("realloc failed\n");
        free(b);
        return 1;
    }
    printf("\nMemory resized using realloc()\n");
    a[3] = 40;
    a[4] = 50;
    for (i = 0; i < 5; i++) {
        printf("a[%d] = %d\n", i, a[i]);
    }

    // 4. free()
    free(a);
    printf("\nMemory allocated by malloc/realloc has been freed.\n");
    free(b);
    printf("Memory allocated by calloc has been freed.\n");

    return 0;
}
```

OUTPUT:

```
ashishreddy@Ashishs-MacBook-Air ~ % nano 8.1.c
ashishreddy@Ashishs-MacBook-Air ~ % gcc 8.1.c -o 8.1
ashishreddy@Ashishs-MacBook-Air ~ % ./8.1
Memory allocated using malloc()
a[0] = 10
a[1] = 20
a[2] = 30

Memory allocated using calloc()
b[0] = 0
b[1] = 0
b[2] = 0

Memory resized using realloc()
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 40
a[4] = 50

Memory allocated by malloc/realloc has been freed.
Memory allocated by calloc has been freed.
```

2) Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#define SIZE (10 * 1024 * 1024) // 10 MB

int main() {
    char *data = malloc(SIZE);
    if (data == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }
    for (int i = 0; i < SIZE; i++) {
        data[i] = 'A';
    }
}
```

```

}
printf("Parent: 10 MB memory allocated.\n");
printf("Parent PID: %d\n", getpid());
pid_t pid = fork();

if (pid < 0) {
    printf("fork() failed\n");
    free(data);
    return 1;
}
if (pid == 0) {
    printf("\nChild PID: %d\n", getpid());
    printf("Child: Memory is initially shared using Copy-on-Write.\n");
    sleep(2);
    printf("Child: Modifying 10 MB of memory...\n");
    for (int i = 0; i < SIZE; i++) {
        data[i] = 'B';
    }
    printf("Child: Memory modification completed.\n");
    sleep(5);
    free(data);
    exit(0);
} else {
    printf("\nParent: Child created using fork().\n");
    printf("Parent: Both processes initially share the same memory
pages.\n");
    sleep(8);
    wait(NULL);
    printf("\nParent: Child process completed.\n");
    free(data);
}
return 0;
}

```

OUTPUT:

```

ashishreddy@Ashishs-MacBook-Air ~ % nano 8.2.c
ashishreddy@Ashishs-MacBook-Air ~ % gcc 8.2.c -o 8.2
ashishreddy@Ashishs-MacBook-Air ~ % ./8.2
Parent: 10 MB memory allocated.
Parent PID: 751

Parent: Child created using fork().
Parent: Both processes initially share the same memory pages.

Child PID: 752
Child: Memory is initially shared using Copy-on-Write.
Child: Modifying 10 MB of memory...
Child: Memory modification completed.

Parent: Child process completed.

```